

Google Colab CLI 小白指南

从远程会话到本地证据链 用真实上游日志管线完成第一次实验

33 项测试通过：2 项未改动的上游测试、25 项管线/边界测试、6 项安全与静态源码检查。真实生成 11 条合成事件、4 种导出文件和含 12 个单元的 Notebook。

实测范围仅为本地 HistoryLogger 与 converter。输入事件及其输出是合成数据，记录中的代码未执行。没有运行完整 CLI、Google 登录、远程 CPU/GPU/TPU 或 CCU 计费。

疑问就是目录

- 01 终端里装了它，算力究竟在哪里？
- 02 已有 Colab 网页，为什么还要 CLI？
- 03 new、exec、run 各负责哪一段？
- 04 脚本、Notebook、REPL 和 console 怎么选？
- 05 本地文件、远程文件与下载结果是什么关系？
- 06 默认认证是 ADC 还是 OAuth2？
- 07 只加 --config，就把实验完全隔离了吗？
- 08 JSONL 历史为什么值得保留？
- 09 导出 ipynb、Markdown、TXT 会丢什么？
- 10 没有登录 Google，怎样做一次真实实验？
- 11 Google One 的 200 CCUs 能直接等于免费 GPU 吗？
- 12 学完以后，怎样知道自己真的掌握了？

阅读路径：先看第 1 问厘清边界，再按五步实验复跑；最后解释导出损失。

这个项目给我什么

1 远程会话管理

按官方源码支持申请、列出、查看、停止 CPU/GPU/TPU 会话；本次未调用远程接口。

2 多种执行入口

支持脚本、Notebook、stdin、REPL 与终端；交互内核和 shell 的通道不同。

3 文件与环境操作

提供上传下载、远程文件管理、依赖安装和 Drive 挂载；均需要相应权限与服务条件。

4 单次任务编排

run 串起申请、执行和尽力收尾；实际代码没有自动下载文件步骤，关键文件需另行保存。--keep 会改变释放责任。

5 本地历史归档

日志查看、事件筛选与多格式导出；本次重点实测上游 history/converter 模块。

4 个具体用途

1 建立实验留痕习惯

用本次离线包练习事件记录、Notebook 导出与字段对比，无需 Google 凭据。

2 整理可复核的故障材料

保留错误事件、原始 JSONL 和版本，避免只交一段被截断的终端输出。

3 为批量脚本准备工作流

有授权和预算后再将参数化分析送到远程；提前明确输入、输出和失败收尾。

4 比较报告格式的代价

同一份历史导出 JSONL、Notebook、Markdown、TXT，判断何时用原始记录、何时用阅读副本。

01 终端里装了它，算力究竟在哪里？

CLI 是遥控器，真正运行代码的是 Colab 远程机器。

命令行界面（CLI）让你用文字命令操作服务。这个项目把申请会话、执行 Python、传文件和释放资源接到终端。安装它不会让本机凭空多出 GPU，也不代表账号一定拿得到某种加速器。

术语落地：运行时（runtime）是承载代码的远程执行环境；会话（session）把本地名字、后端分配和访问状态关联起来。

场景：你在笔记本上编辑分析脚本，CLI 把代码送到远程内核；图表和文件还需按需要取回。本次只实测本地日志模块，没有申请远程机器。



先分清“在哪里发命令”和“在哪里算”，本次实验停留在本地。

依据：S1、S6

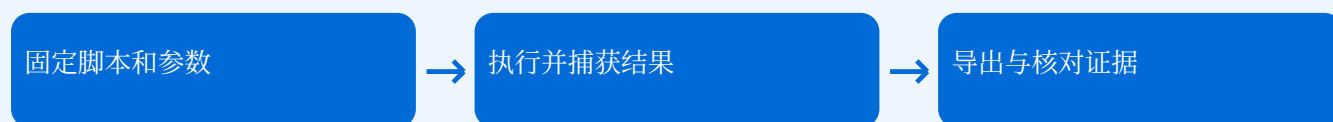
02 已有 Colab 网页，为什么还要 CLI？

把重复的手工操作，变成可检查的命令步骤。

网页适合交互探索；CLI 适合把本地文件、参数、执行与取回过程接进工作流。它也能给 AI 代理提供操作接口，但自动化程度不会扩大权限、额度或服务允许的用途。是否适合长期无人值守，还要看认证、配额、失败处理和释放证据。

术语落地：工作流是有输入、步骤、输出与失败分支的一组操作。自动化只是减少重复点击，不是可靠性证明。

场景：同一个脚本换三个参数时，固定版本和记录每次输出比凭记忆重做更可靠。先在本地学会留存证据，再考虑经过授权的远程运行。



复用“工具调用生命周期”：发出请求之后，还要判断成功与收尾。

依据：S1、S7

03 new、exec、run 各负责哪一段？

new 申请，exec 使用已有会话，run 把一次任务串起来。

new 申请运行时并记住会话；exec 把代码送入所选会话；stop 释放会话。run 则为脚本申请新会话、传递参数、执行并尝试收尾。--keep 会跳过自动释放，必须另行管理。程序的 finally 收尾设计，并不能证明每次网络故障后资源都已释放。

术语落地：生命周期指资源从申请到使用再到释放的全过程；幂等是重复操作仍能得到可预测结果，本次没有验证远程操作的幂等性。

场景：一次性图片处理可用单次任务思路；需要多次交互调试则复用会话。选择前先确定失败时如何查状态、保结果、停资源。



源码显示 run 有 finally 清理路径；本次没有进行远程释放验证。

依据：S1、S7

04 脚本、Notebook、REPL 和 console 怎么选？

Python 内核与终端 shell 是两条不同的执行通道。

exec 支持本地 Python 文件、Notebook 和标准输入；REPL 是逐条输入 Python 的交互循环。console 提供远程终端 shell，绕过 Jupyter 内核。TTY 表示交互式终端，管道输入不是 TTY，因此结束输入和退出行为需要单独处理。

术语落地：内核（kernel）执行 Notebook 的代码单元；shell 解释系统命令。两者共享机器并不等于使用相同协议或语法。

场景：导出器给 source=piped 且不以 ! 开头的代码加 %%bash。但管道输入的 Python REPL 也使用 piped 标记，存在被误当 shell 的风险，重跑前必须检查。



piped 是来源标签，不是可靠语言识别；不要直接重跑导出的单元。

依据：S6、S9、S13

05 本地文件、远程文件与下载结果是什么关系？

传代码与传数据是两件事；远程成功不等于本地已保存。

本地脚本可以被读取后发送给内核，不必先作为文件上传。脚本引用的数据文件却仍须在远程可见的位置准备好。upload、download 与远程文件管理负责另一条数据通路。运行时可能被回收，关键输出应按授权取回并检查大小、格式或摘要。

术语落地：数据面负责实际字节与结果；控制面负责申请、选择、停止等指令。沿用这个区分，可以减少“命令成功但文件没到手”的误判。

场景：日志里写着 download，并不能证明某份真实模型已下载。本次使用明确标为虚构的文件事件，只验证日志表示与格式转换。



历史事件是操作记录；真实文件是否存在需要独立证据。

依据：S1、S8

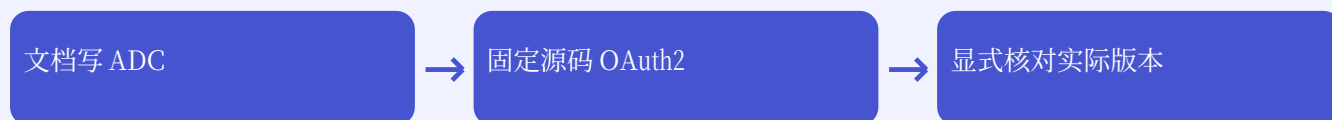
06 默认认证是 ADC 还是 OAuth2？

这个固定版本里，代码默认 OAuth2；README 写成 ADC。

OAuth2 是让应用获得用户授权的机制；ADC 是 Google 库寻找应用默认凭据的约定。固定提交的 cli.py 与 common.py 都选择 OAUTH2，README 的默认 adc 描述与之冲突。学习时应保留差异，部署前检查实际版本的帮助和源码，不把文档一句话当运行事实。

术语落地：认证回答“你是谁”，授权回答“可以做什么”。工具存在、账号登录和服务权益是不同层次。

场景：一个看似查询信息的命令也可能刷新凭据或访问网络。本实验直接导入日志与导出模块，并用进程内网络阻断，不触发任何 Google 登录。



截至本次固定提交可定位到具体文件；不能外推到所有未来版本。

依据：S1、S3、S4

07 只加 --config，就把实验完全隔离了吗？

--config 只改会话状态文件，其他路径仍可能跟着 HOME。

common.py 的会话存储接受 config_path，但历史记录器与设置存储分别使用默认路径；日志路径也独立处理。因此另指定 sessions.json 不足以隔离所有本地状态。本次在项目内安排 HOME、缓存和临时目录，并绕开整套 CLI 启动与认证入口。

术语落地：隔离边界要由实际路径与访问行为证明。进程内 socket 阻断能拦住测试进程的联网尝试，不等于操作系统级安全沙箱。

场景：换一本“会话账本”，不等于把“历史档案”和“设置抽屉”也搬走。若导入模块时就计算 HOME，必须在导入前设好进程环境。

会话文件 --config



历史和设置 HOME



逐类确认边界

实验设置只作用于项目与当前进程，没有修改全局配置。

依据：S4、S8、S3

08 JSONL 历史为什么值得保留？

一行一个事件，让机器可以重新读取完整结构。

JSONL 每行保存一个可独立解析的 JSON 值；这里每行是事件对象。HistoryLogger 追加 timestamp、event_type 和事件数据，按会话保存。get_history 逐行读回，list_sessions 列出有日志的会话。它不是不可篡改审计系统，缺失或损坏记录仍要检查。

术语落地：事件（event）描述某个时刻发生的动作及相关数据；历史快照记录事实的表示，不能代替执行本身。

场景：把“代码是什么”和“输出是什么”写进 execution 事件，随后能导出教学 Notebook。人工写入一个成功输出，只能证明导出能力，不能证明计算真的发生。

结构化虚构事件



真实 logger 追加



JSONL 原样读回

本实验的输入是合成事件，实际执行的是上游日志代码。

依据：S8、S11

09 导出 ipynb、Markdown、TXT 会丢什么？

格式方便程度不同，保留的信息也不同。

JSONL 导出逐事件保存结构；ipynb 把部分事件变成代码或说明单元。Markdown 着重代码与 text 输出；TXT 对 execution 主要写代码。导出器还会把含 text 的输出统一写成 stdout，并忽略部分事件类型。因此可读版不是完整原始记录的替代品。

术语落地：序列化把结构变成可保存的字节；投影只选择原数据的一部分。Notebook 和 Markdown 在这里更接近面向阅读的投影。

场景：当输出没有 text/data 且 output_type=error 时，错误会映射到 Notebook；Markdown 可能漏掉这些详情。原始 JSONL 应与可读报告一起保存。



漂亮的报告不是无损备份，导出之后还要比较关键字段。

依据：S9

10 没有登录 Google，怎样做一次真实实验？

运行真实本地模块，明确把远程服务留在验证范围之外。

程序可拆成多层。HistoryLogger 和 converter 的本地路径，可以用固定上游源码、可控输入和独立断言运行。网络阻断与项目内目录减少意外触达账号的风险。这样能真实验证这条管线，却不能证明会话申请、GPU、计费或某篇帖文的运行结论。

术语落地：测试替身是替代外部系统的受控对象；合成夹具是专门设计的输入。这里没有改写 logger 或 converter，只把输入标为合成数据。

场景：同一输入分别导出四种格式，再用 nbformat 校验 Notebook 结构，检查 shell 包装、错误输出与事件数量。这比只截一张“运行成功”图更可复核。



真实代码执行与真实云端执行必须分别举证。

依据：S8、S9、S11

11 Google One 的 200 CCUs 能直接等于免费 GPU 吗？

权益条件、账号实际资格、硬件可用性，需要分开核实。

2026-10-05 查阅的 Google One 帮助页列出 AI Pro 的 200 CCUs，并对 Colab 权益注明网页使用、成年人、家庭方案管理员、排除试用等条件。CCU 是计算额度计量，不是某款 GPU 的固定时长或可用性保证。用户自己的订阅、余额与 CLI 资格未知。

术语落地：额度（quota/credit）限制可消费资源；权益（entitlement）决定可用功能。服务端限制与资源供给还会随时间变化。

场景：即使看到 200 这个数字，也要在自己的账号界面核实当前规则与余额。不要由 CLI 支持某参数推导出账号一定能申请对应硬件。



官方服务条件见来源 G1、G2；本次没有登录或消费任何额度。

依据：S1

12 学完以后，怎样知道自己真的掌握了？

能复跑、能解释差异、能说清没验证的部分。

先固定源码版本与环境，再运行主实验、读取可见产物、对比导出损失，并检查测试与断言结果。最后把新概念接到已有“产物留痕”和“托管会话”知识上。真实的失败边界和复跑记录，比泛泛的功能清单更能检验理解。

术语落地：可复现不要求随机 cell id 和时间戳逐字一致，而要求方法、输入、依赖与可观察语义能够再次核对。

场景：先回答三个问题：哪份文件是原始事件？哪份方便阅读？哪条证据证明发生过真正的远程运算？本次第三问的答案是：没有这样的证据。



下一步如果要接真实 Colab，需另行确认账号、授权、预算和退出方案。

依据：S8、S9、S11

五步主实验

先解压 03-exercise.zip，进入 google_colab_cli_offline_experiment 目录；以下全部命令都在该目录运行。需要 Python 3.12 或更高版本。第 2 步从官方 PyPI 联网安装，之后运行的主实验离线。长命令末尾的反斜杠表示续行。

01 创建隔离环境

目的：在新解压目录创建项目专用虚拟环境

```
python3 -m venv .venv
```

真实执行时间：2026-10-05 05:33:41 至 05:33:43 UTC

实现了什么：本地.venv创建成功

验证标准与实际结果：退出码0；虚拟环境路径位于新解压目录

02 安装已验证依赖

目的：从官方PyPI安装本次Linux依赖版本；此准备步骤需要联网

```
.venv/bin/python -m pip --isolated install --no-cache-dir \
--index-url https://pypi.org/simple \
-r requirements-tested-linux.txt
```

真实执行时间：2026-10-05 05:33:43 至 05:34:04 UTC

实现了什么：nbformat5.10.4和锁定版本依赖安装成功

验证标准与实际结果：退出码0；完整安装输出已保留；无全局安装

仅写入解压目录的 .venv、runtime、outputs。主脚本会设置本地 HOME/缓存/临时目录并阻断测试进程联网；这不是 OS 级沙箱。重跑会覆盖本练习生成的输出，保留旧结果时请先复制整个目录。

继续复跑并核对证据

03 核验上游源码

目的：确认运行的是固定版本的真实上游模块

```
.venv/bin/python verify_sources.py
```

真实执行时间：2026-10-05 05:34:04 至 05:34:04 UTC

实现了什么：14个上游文件SHA-256一致

验证标准与实际结果：source_manifest.json与原始核验日志

04 运行离线实验

目的：运行上游HistoryLogger/converter、边界和负对照并输出四种格式

```
.venv/bin/python run_experiment.py
```

真实执行时间：2026-10-05 05:34:04 至 05:34:04 UTC

实现了什么：33/33通过；四种导出完成；无意外网络尝试

验证标准与实际结果：outputs/summary.json、tests_raw.txt与四种导出文件

继续复跑并核对证据

05 核对落盘产物

目的：检查实际事件数、notebook结构、文件哈希和格式信息差异

```
.venv/bin/python inspect_results.py
```

真实执行时间：2026-10-05 05:34:04 至 05:34:04 UTC

实现了什么：11事件；12单元/6代码；输出哈希一致；错误和富内容在Markdown缺失

验证标准与实际结果：inspect_results.py打印的JSON与实际导出文件相互印证

打开产物 看见差异

inspect_results.py 的实际核对结果

```
jsonl_event_count: 11
notebook_cell_count: 12
notebook_code_cell_count: 6
notebook_has_error_object: true
markdown_has_error_object: false
all_recorded_output_hashes_match: true
```

先用文本编辑器打开 outputs/fictional-demo.jsonl：每行是一个事件，终止事件也在。

对比同目录的 .ipynb 与 .md：Notebook 保存了错误对象，Markdown 没有。不要点击运行其中的代码。

12 个 Notebook 单元不等于 12 次计算；其中 6 个是代码单元，它们由合成历史生成，并未执行。

真实项目的日志可包含代码、输入回答和输出。分享前先检查敏感内容；本次内容全部虚构。

真正得到哪些证据

- 通过不等于没有缺点：测试确认了元数据可覆盖、损坏 JSONL 报错，以及导出器的已有信息损失；没有修改上游代码修复这些行为。
- JSONL 保留 11 条输入事件；Notebook 有 12 个单元，其中 6 个代码单元，但没有保存终止事件，stderr 标签和 execute_result 类型也会改变。
- Markdown 没有错误对象和富 JSON 输出，TXT 没有 execution 的输出；原始事件和人读导出要一起保留。
- piped 标记不是可靠语言识别；Python 也可能被加 %%bash。导出的代码没有执行，重放前需先审查。
- 网络负对照被拦截，测试没有非预期网络尝试；这不涵盖之前的 PyPI 依赖安装。完整 CLI 和全部远程集成测试未运行。
- 新解压目录、全新虚拟环境中的五步复跑通过；重复执行仍是同一组 33 用例，不累加成更大的测试数。

已有知识如何复用

- 历史公开查重范围：固定索引 394 条元数据；全文读取 10 篇，余 384 篇仅元数据。此范围不能替代最新知识库全文查重。
- 复用已有产物留痕、控制面与数据面、会话资源概念；其他产品的认证、计费或持久化细节只作类比，不能当成 Colab 的事实。
- 可复用的方法：声明与强制权限分层，有限测试不外推，来源主张与实际验证分开。
- 本次公开整理不新增概念：JSONL 有损导出、Notebook 文件与运行时分层分别增补已有日志、产物留痕与 Checkpoint 笔记；来源标签歧义保留为项目案例。

知识整理建议

知识笔记保留定义与证据边界；结合最新知识库做同义查重，优先增补已有概念，相关概念互链。

实际运行：2026-10-05T05:33:41.644629+00:00 至 2026-10-05T05:34:04.483569+00:00

本次实跑：Linux, Python 3.12.14, nbformat 5.10.4；完整依赖版本在 requirements-tested-linux.txt。Windows 未实跑；官方完整 CLI 仅声明支持 Linux/macOS。

来源与复核入口

所有上游代码固定到 a84e094c67544e70d88649ba2d2a1d48511b3af7。直接执行真实 history 与 converter 模块；没有安装或运行完整 colab 命令入口。每条实验结论都限于本地受控输入。

S1 项目定位与命令入口

README.md

S2 实际依赖与 Python 版本

pyproject.toml

S3 CLI 全局参数与默认认证

src/colab_cli/cli.py

S4 会话状态与独立存储路径

src/colab_cli/common.py

S5 会话管理与生命周期

docs/01_session_management.md

S6 执行模式与交互机制

docs/02_execution_and_interactive.md

S7 一次性任务与释放路径

docs/05_run_command.md

S8 真实历史记录器

src/colab_cli/history.py

S9 真实格式导出器

src/colab_cli/converter.py

S10 日志筛选与导出口

src/colab_cli/commands/utility.py

S11 上游历史记录单元测试

tests/test_history.py

S12 脚本任务的实际执行和收尾

src/colab_cli/commands/run.py

S13 REPL 的 piped 来源标记

src/colab_cli/commands/execution.py

S14 源码许可证

LICENSE

上方来源标题可点击，均指向固定提交。完整 URL、许可证与本地证据清单同时保存在 HTML 和练习 ZIP。

G1 Google One 权益；G2 Colab 资源限制

2026-10-05 查阅。200 CCUs 有资格条件；用户权益、余额与 CLI 适用性未知。资源与服务规则可变化。

产物检查

PDF 由 ReportLab 直接生成，按 A4 排版并逐页渲染检查。HTML 有 12 个疑问和 12 张内联 SVG，仅通过结构检查，没有声明浏览器视觉验收。练习 ZIP 保留所需上游源码、许可、版本指纹、测试脚本、依赖清单与实测结果；不包含完整虚拟环境。

上游许可 Apache-2.0。引用与随包源码保留原始许可证及版权声明；原创讲解、虚构数据与测试脚本不代表上游官方保证。